Homework 2 — Word2Vec

Alexandre Allauzen and Florian Le Bronnec

March 17, 2026

1 Introduction

You are expected to implement from scratch the Word2Vec model using PyTorch.

Word2Vec is a contrastive representation learning model that learns word embeddings. The list below is a formulation of the Word2Vec algorithm that you are expected to implement.

Algorithm Description

1. Sample a word $w := w_i$ from the dataset $\mathcal{D}$ from a random document, where i is the index of the word in this document.

2. Take the words surrounding w in a local window of radius R in the document, they will make the positive context:
$$C^+ = [w_{i-R}, \dots, w_{i-1}, w_{i+1}, \dots, w_{i+R}].$$
This yields $2R$ positive examples.

3. The usual implementation of Word2Vec uses more negative examples than positive ones. This is parametrized by the integer factor K: from the vocabulary $\mathcal{V}$, sample $2KR$ random words which will make the negative examples, C^-.

4. Map the word w to embedding $\boldsymbol{w} \in \mathbb{R}^d$ using embedding table E^w.

5. Map each word in C^+ and C^- to vectors $\boldsymbol{C}^+ \in \mathbb{R}^{2R\times d}$, $\boldsymbol{C}^- \in \mathbb{R}^{2R\times d}$ through the context embeddings table E^C.

6. Similarity score between the word w and the context words: for a context word $c \in C^+ \cup C^-$, the similarity score is given by $\sigma(\boldsymbol{c}^{\mathrm{T}}\boldsymbol{w})$ where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid function.

7. We therefore have a model M that maps a word w, another word c from a positive (resp. negative) context $c \in C^+$ (resp. $c \in C^-$) to a scalar $\sigma(\boldsymbol{c}^{\mathrm{T}}\boldsymbol{w})$. The loss will be the following function for binary classification:
$$L(M(w,c)) = -\mathbf{1}_{c\in C^+} \log(\sigma(\boldsymbol{c}^{\mathrm{T}}\boldsymbol{w})) - \mathbf{1}_{c\in C^-} \log(1-\sigma(\boldsymbol{c}^{\mathrm{T}}\boldsymbol{w})), \tag{1}$$
where $\mathbf{1}_{c\in C^+}$ (resp. $\mathbf{1}_{c\in C^-}$) is equal to 1 if $c \in C^+$ (resp. $c \in C^-$) and otherwise is equal to 0.

2 Preliminary Questions

1. To compute the similarity between a word and a context, we use the similarity $\sigma(\boldsymbol{c}^{\mathrm{T}}\boldsymbol{w})$. Since, we want to minimize the loss (1):

   (a) For $c \in C^+$, should we maximize or minimize $\sigma(\boldsymbol{c}^{\mathrm{T}}\boldsymbol{w})$?

   (b) Same question for $c \in C^-$.

   Give a simple geometrical interpretation.

2. One of the first applications to contrastive learning has been presented by Chopra, Hadsell, and LeCun [1].

   (a) By reading the introduction of this article, explain briefly contrastive learning.

   In page 3 of this article, you will find the following function:

$$\mathcal{L}(W, (Y, X_1, X_2)^i) = (1 - Y)L_G(E_W(X_1, X_2)^i) + YL_I(E_W(X_1, X_2)^i). \tag{2}$$

   This is similar to what we are doing. In relation with the algorithm described above in Section 1, explain:

   (a) What is the analog of Y?

   (b) What is the analog of E_W? Why is it not the same?

   (c) What are the analogs of L_G and L_I?

3 Implementation

This section aims at guiding you through the construction of your own Word2Vec model. You are required to follow these instructions. For readability purposes, when possible, comment which part of the code refers to which question. Make sure you understand the experimental setup described in Section 1.

3.1 Data Preprocessing

1. The first cell of the notebook should be used to load a preprocessing function for the IMDB dataset containing movie reviews. Apply a similar preprocessing than the ones used during previous lab sessions to get a dataset (with the same tokenizer) with a train and a validation split, with one columns `input_ids` (list of int). We do not use the annotated labels in this section, since Word2Vec is an unsupervised model.

2. Write a function `extract_words_contexts`. It should retrieve all pairs of valid (w, C^+) from a list of ids representing a text document. It takes the radius R as an argument. Its output is therefore two lists:

   - The first one contains the ids of w.
   - The second one contains the list of ids of C^+, within the local window, corresponding to w.

   Make sure that every C^+ has the same size (i.e., the same number of ids). Explain clearly how you handle the borders (any strategy can be valid as as long as it is justified).

3. Write a function `flatten_dataset_to_list` that applies the function `extract_words_contexts` on a whole dataset.

4. Apply the function to your initial `document_train_set` and `document_valid_set`, and get the corresponding flattened lists.

5. Embed these lists in two valid PyTorch Dataset, like in HW1, call them `train_set` and `valid_set`.

6. Write a `collate_fn` function that adds the negative context to the batch. It should be parametrized by the scaling factor K. The output of `collate_fn` should be a Python dictionary, with three keys:

   - `word_id`,
   - `positive_context_ids`,
   - `negative_context_ids`.

   Make sure that each value of the dictionary is a valid `torch.Tensor`. *Hint:* for the negative context, simply randomly sample from the whole vocabulary set.

7. Wraps everything in a `DataLoader`, like in HW1.

8. Make 2 or 3 iterations in the `DataLoader` and print R, K and the shapes of all the tensors in the batches (let the output be visible).

3.2 Model

1. Write a model named `Word2Vec` which is a valid `torch.nn.Module` (i.e., write a class that inherits from `torch.nn.Module`), and implement the Word2Vec model. It should be parametrized by the vocabulary size and the embeddings dimension. Use the module `torch.nn.Embedding`.

2. Train the model. The training should be parametrized by the batch size B, and the number of epochs E.

3. Validate its accuracy on the test set.

4. Write a function `save_model` that saves the model's embeddings in a file. The file name should be formated like:

   `"model_dim-<d>_radius-<R>_ratio-<K>-batch-<B>-epoch-<E>.ckpt".`

5. Once you have a working code, you can launch a bigger training, using more documents, if it does not take too much time.

4 Few-Shots Classification Task

In this section you will experiment with the actual classification task, augmented with your Word2Vec model. Here we will mimick a setup of **few-shots learning**:

- We assume we have only a few labeled examples (64 here, but you can modify this number).

- We will try to classify the reviews of the validation set, using the embeddings learned by the Word2Vec model.
- Compare the results with the model without this initialization. Use the supplied notebook `imdb_few_shots.ipynb`.

1. Write a function `load_model` that takes a path to a saved Word2Vec embeddings (with the previous formatting) and loads the checkpoint the embeddings directly to the WordEmbedClassifier.
2. Train the model, initialized with these emebeddings.
3. Compare the results with the model without this initialization.
4. Make a small ablation study on the influence of some parameters of the Word2Vec model on the classification task. Analyze the results.

General Instructions

Submission. You are expected to submit three files:

1. A **written report in PDF**.
2. **Code for the Word2Vec model** in a **Jupyter notebook** from the supplied template.
3. Another **Jupyter notebook** with the application of the model to few-shots classification.

Report guidelines.

- The report should contain the answers to the questions asked in Section 2.
- It should also feature a general summary on your implementation.
- In particular, you are expected to justify each choice you made in the implementation of the model that is not imposed.
- You should also report and analyze the results you obtained (with plots and/or tables).
- It should contain a small conclusion on what you learned by doing this HW.
- The number of pages should be **at most 3**.

Coding guidelines. The only requirement is that the notebooks you submit do not return any error when running on Google Colab. Any effort made to make the code readable and easy to follow will be valued. Here are some hints.

- This does not mean to add comments to every line of the code, but to write more understandable code and to comment on the parts when it is necessary.
- Choose meaningful variable names.
- Write modular code with meaningful function names. If necessary, put a quick docstring in your functions indicating what it does.

- Print intermediary results, like shapes or a small number of samples in explanative cells.
- Typing is not mandatory, though this is a general good practice and can help for the readability.
- Use the `Black` code style on your final notebook (see `https://black.readthedocs.io/en/stable/getting_started.html`).

References

[1] Sumit Chopra, Raia Hadsell, and Yann LeCun. Learning a similarity metric discriminatively, with application to face verification. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, volume 1, pages 539–546. IEEE, 2005.